

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Sistemas Operativos 2



Práctica 1 - Tuberías

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Septiembre de 2026.

Problema 1 (Comunicación unidireccional y Bidireccional)

```
=====
PROBLEMA 1: SISTEMA DE PAGO EN LINEA CON VERIFICACION DE ESTADO
=====

Bienvenido al sistema de pagos de telefonía.
Opciones de verificación previa:
  1. Realizar verificación de estado del canal
  2. Omitir verificación e ir directamente al pago
Seleccione una opción (1 o 2): 1

--- FASE 1: VERIFICACION DE CANAL (UNIDIRECCIONAL) ---
Ingrese una palabra para probar la conexión del canal: telecomunicaciones
[PADRE - PID 24723]: Enviando palabra "telecomunicaciones" al hijo por la tubería...

[HIJO - PID 26142]: Recibi la palabra: "telecomunicaciones"
[HIJO - PID 26142]: Inversión manual completada: "senoicacinumocet"
[HIJO - PID 26142]: >>> CONFIRMACION: El canal está ACTIVO y respondiendo. <<<

--- FASE 2: PROCESAMIENTO DEL PAGO (BIDIRECCIONAL) ---
Ingrese su número de tarjeta (entre 1000 y 9999): 2000
[PADRE - PID 24723]: Enviando tarjeta 2000 al servidor de pagos (hijo)...
[HIJO - PID 26237]: Validando número de tarjeta recibido: 2000
[HIJO - PID 26237]: Evaluación: PAGO_APROBADO. Enviando veredicto al padre...

=====
RESULTADO FINAL DEL SISTEMA (PADRE): PAGO_APROBADO
Estado: Transacción completada con éxito.
=====

Presione ENTER para volver al menú principal...█
```

```
=====
PROBLEMA 1: SISTEMA DE PAGO EN LINEA CON VERIFICACION DE ESTADO
=====

Bienvenido al sistema de pagos de telefonía.
Opciones de verificación previa:
  1. Realizar verificación de estado del canal
  2. Omitir verificación e ir directamente al pago
Seleccione una opción (1 o 2): 2

[*] Verificación omitida por el usuario. Procediendo directamente al pago...

--- FASE 2: PROCESAMIENTO DEL PAGO (BIDIRECCIONAL) ---
Ingrese su número de tarjeta (entre 1000 y 9999): 1999
[PADRE - PID 24723]: Enviando tarjeta 1999 al servidor de pagos (hijo)...
[HIJO - PID 27256]: Validando número de tarjeta recibido: 1999
[HIJO - PID 27256]: Evaluación: PAGO_RECHAZADO. Enviando veredicto al padre...

=====
RESULTADO FINAL DEL SISTEMA (PADRE): PAGO_RECHAZADO
Estado: La transacción fue declinada por fondos insuficientes o rechazo.
=====

Presione ENTER para volver al menú principal...█
```

Preguntas de análisis

- **¿Qué ocurriría si el hijo intentara leer de la tubería antes de que el padre haya escrito algo?**

La llamada al sistema `read()` sobre un descriptor de archivo asociado a una tubería sería bloqueante por defecto. Si el búfer interno del pipe en el núcleo está vacío y aún existe al menos un descriptor de escritura abierto para esa tubería en el sistema, el proceso hijo que invoca `read()` es suspendido inmediatamente por el planificador del sistema operativo, pasando del estado Running al estado Sleeping / Blocked (esperando eventos de E/S).

El proceso hijo no consumirá ciclos de CPU mientras espera. Permanecerá en ese estado hasta que ocurra uno de dos eventos:

- El proceso padre escriba datos en la tubería mediante `write()`, momento en el cual el kernel despierta al hijo y `read()` retorna la cantidad de bytes leídos.
 - Todos los procesos que posean descriptores de escritura abiertos los cierren (`close()`), momento en el cual `read()` desbloquea inmediatamente al hijo y retorna 0 (indicando la condición de fin de archivo o EOF).
- **La segunda fase requiere que el hijo envíe una respuesta de vuelta al padre. ¿Por qué no es suficiente con la misma tubería usada en la verificación?**

Las tuberías anónimas tradicionales en UNIX son canales unidireccionales (Half-Duplex). Cuentan con un extremo dedicado exclusivamente a la lectura (`fd[0]`) y otro a la escritura (`fd[1]`), estructurados internamente como una cola FIFO en memoria del núcleo.

Aunque tras la llamada a `fork()` tanto el padre como el hijo heredan y comparten ambos extremos, una sola tubería no posee mecanismos para filtrar o enrutar mensajes hacia un destinatario específico. La tubería simplemente almacena una secuencia continua de bytes.

Para lograr una comunicación BIDIRECCIONAL confiable (Full-Duplex lógico), se requieren forzosamente dos tuberías independientes:

- Tubería A (Padre -> Hijo): El padre escribe y el hijo lee.
 - Tubería B (Hijo -> Padre): El hijo escribe y el padre lee.
- **¿Qué problema concreto surgiría si ambos procesos intentaran leer y escribir sobre la misma tubería?**

Surgiría una condición de carrera crítica (Race Condition) y un riesgo inminente de interbloqueo (Deadlock) y corrupción de flujo de datos.

Caso concreto: Suponiendo que el padre escribe el número de tarjeta en la tubería para que el hijo lo valide, e inmediatamente después el padre ejecuta `read()` en esa misma tubería a la espera de la respuesta. Como el sistema operativo atiende las lecturas de forma atómica y no distingue la identidad del proceso que colocó los datos:

- Es muy probable que el propio proceso padre lea y consuma inmediatamente el número de tarjeta que acaba de escribir, antes de que el hijo tenga la oportunidad de ser planificado en la CPU.
- Como consecuencia, el padre procesará su propio mensaje como si fuera la respuesta del hijo y esto provocará datos corruptos o errores lógicos.

- Cuando el hijo finalmente sea planificado y ejecute read(), la tubería estará vacía, dejándolo bloqueado indefinidamente esperando un dato que ya fue consumido, produciendo un interbloqueo permanente (Deadlock).

Problema 2 (Productor - Consumidor)

```
=====
PROBLEMA 2: BODEGA DE CAMISAS (PRODUCTOR - CONSUMIDORES EN PARALELO)
=====

Opciones de ingreso de los 20 pedidos:
  1. Ingresar manualmente los 20 pedidos (valores entre 1 y 100)
  2. Generar aleatoriamente los 20 pedidos (para evaluacion rapida)
Seleccione modalidad (1 o 2): 1

--- Registro de pedidos del dia (1 a 100 camisas por pedido) ---
Pedido #01 - Cantidad de camisas (1-100): 20
Pedido #02 - Cantidad de camisas (1-100): 14
Pedido #03 - Cantidad de camisas (1-100): 15
Pedido #04 - Cantidad de camisas (1-100): 4
Pedido #05 - Cantidad de camisas (1-100): 33
Pedido #06 - Cantidad de camisas (1-100): 223
[!] Error: El valor debe estar entre 1 y 100. Intente de nuevo.
Pedido #06 - Cantidad de camisas (1-100): 32
Pedido #07 - Cantidad de camisas (1-100): 32
Pedido #08 - Cantidad de camisas (1-100): 44
Pedido #09 - Cantidad de camisas (1-100): 99
Pedido #10 - Cantidad de camisas (1-100): 10
Pedido #11 - Cantidad de camisas (1-100): 21
Pedido #12 - Cantidad de camisas (1-100): 23
Pedido #13 - Cantidad de camisas (1-100): 31
Pedido #14 - Cantidad de camisas (1-100): 15
Pedido #15 - Cantidad de camisas (1-100): 235
[!] Error: El valor debe estar entre 1 y 100. Intente de nuevo.
Pedido #15 - Cantidad de camisas (1-100): 23
Pedido #16 - Cantidad de camisas (1-100): 42
Pedido #17 - Cantidad de camisas (1-100): 51
Pedido #18 - Cantidad de camisas (1-100): 17
Pedido #19 - Cantidad de camisas (1-100): 27
Pedido #20 - Cantidad de camisas (1-100): 10

[COORDINADOR]: Registro de los 20 pedidos finalizado.
[COORDINADOR]: Apertura de la banda transportadora (tuberia) y despliegue de estaciones...
[COORDINADOR]: Enviando inmediatamente los 20 pedidos por la banda transportadora...

[ESTACION 2 - PID 29852]: Despachando pedido de 20 camisas. (Acumulado: 1 pedidos / 20 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 14 camisas. (Acumulado: 1 pedidos / 14 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 15 camisas. (Acumulado: 2 pedidos / 35 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 4 camisas. (Acumulado: 2 pedidos / 18 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 33 camisas. (Acumulado: 3 pedidos / 68 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 32 camisas. (Acumulado: 3 pedidos / 50 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 32 camisas. (Acumulado: 4 pedidos / 100 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 44 camisas. (Acumulado: 4 pedidos / 94 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 99 camisas. (Acumulado: 5 pedidos / 199 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 10 camisas. (Acumulado: 6 pedidos / 209 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 21 camisas. (Acumulado: 5 pedidos / 115 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 23 camisas. (Acumulado: 7 pedidos / 232 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 31 camisas. (Acumulado: 6 pedidos / 146 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 15 camisas. (Acumulado: 8 pedidos / 247 camisas)
```

```
[ESTACION 1 - PID 29851]: Despachando pedido de 31 camisas. (Acumulado: 6 pedidos / 146 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 15 camisas. (Acumulado: 8 pedidos / 247 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 23 camisas. (Acumulado: 9 pedidos / 270 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 42 camisas. (Acumulado: 7 pedidos / 188 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 51 camisas. (Acumulado: 10 pedidos / 321 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 17 camisas. (Acumulado: 8 pedidos / 205 camisas)
[ESTACION 2 - PID 29852]: Despachando pedido de 27 camisas. (Acumulado: 11 pedidos / 348 camisas)
[ESTACION 1 - PID 29851]: Despachando pedido de 10 camisas. (Acumulado: 9 pedidos / 215 camisas)
```

```
=====
REPORTE FINAL: ESTACION DE EMPAQUE 2 (PID: 29852)
=====
```

```
Total de pedidos procesados : 11 pedidos
Total de camisas despachadas: 348 unidades
=====
```

```
=====
REPORTE FINAL: ESTACION DE EMPAQUE 1 (PID: 29851)
=====
```

```
Total de pedidos procesados : 9 pedidos
Total de camisas despachadas: 215 unidades
=====
```

[COORDINADOR]: Ambas estaciones han terminado. La banda transportadora quedo vacia.

Presione ENTER para volver al menu principal...

```
=====
PROBLEMA 2: BODEGA DE CAMISAS (PRODUCTOR - CONSUMIDORES EN PARALELO)
=====
```

Opciones de ingreso de los 20 pedidos:

1. Ingresar manualmente los 20 pedidos (valores entre 1 y 100)
2. Generar aleatoriamente los 20 pedidos (para evaluacion rapida)

Seleccione modalidad (1 o 2): 2

--- Generando 20 pedidos aleatorios [1 - 100] ---

Pedido #01: 41 camisas	Pedido #02: 58 camisas	Pedido #03: 64 camisas	Pedido #04: 4 camisas
Pedido #05: 99 camisas	Pedido #06: 43 camisas	Pedido #07: 93 camisas	Pedido #08: 58 camisas
Pedido #09: 94 camisas	Pedido #10: 81 camisas	Pedido #11: 73 camisas	Pedido #12: 85 camisas
Pedido #13: 66 camisas	Pedido #14: 51 camisas	Pedido #15: 26 camisas	Pedido #16: 76 camisas
Pedido #17: 91 camisas	Pedido #18: 3 camisas	Pedido #19: 2 camisas	Pedido #20: 96 camisas

[COORDINADOR]: Registro de los 20 pedidos finalizado.

[COORDINADOR]: Apertura de la banda transportadora (tuberia) y despliegue de estaciones...

[COORDINADOR]: Enviando inmediatamente los 20 pedidos por la banda transportadora...

```
[ESTACION 2 - PID 28835]: Despachando pedido de 41 camisas. (Acumulado: 1 pedidos / 41 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 58 camisas. (Acumulado: 1 pedidos / 58 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 64 camisas. (Acumulado: 2 pedidos / 105 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 4 camisas. (Acumulado: 2 pedidos / 62 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 99 camisas. (Acumulado: 3 pedidos / 161 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 43 camisas. (Acumulado: 3 pedidos / 148 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 93 camisas. (Acumulado: 4 pedidos / 254 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 58 camisas. (Acumulado: 4 pedidos / 206 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 94 camisas. (Acumulado: 5 pedidos / 348 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 81 camisas. (Acumulado: 6 pedidos / 429 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 73 camisas. (Acumulado: 5 pedidos / 279 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 85 camisas. (Acumulado: 7 pedidos / 514 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 66 camisas. (Acumulado: 6 pedidos / 345 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 51 camisas. (Acumulado: 7 pedidos / 396 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 26 camisas. (Acumulado: 8 pedidos / 540 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 76 camisas. (Acumulado: 9 pedidos / 616 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 91 camisas. (Acumulado: 8 pedidos / 487 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 3 camisas. (Acumulado: 9 pedidos / 490 camisas)
[ESTACION 1 - PID 28834]: Despachando pedido de 2 camisas. (Acumulado: 10 pedidos / 618 camisas)
[ESTACION 2 - PID 28835]: Despachando pedido de 96 camisas. (Acumulado: 10 pedidos / 586 camisas)
```

```
=====
REPORTE FINAL: ESTACION DE EMPAQUE 1 (PID: 28834)
=====
Total de pedidos procesados : 10 pedidos
Total de camisas despachadas: 618 unidades
=====

=====
REPORTE FINAL: ESTACION DE EMPAQUE 2 (PID: 28835)
=====
Total de pedidos procesados : 10 pedidos
Total de camisas despachadas: 586 unidades
=====

[COORDINADOR]: Ambas estaciones han terminado. La banda transportadora quedo vacia.
Presione ENTER para volver al menu principal...[]
```

Preguntas de análisis

- **¿Es posible predecir de antemano cuántos pedidos procesará cada estación? Relaciona tu respuesta con el concepto de race condition.**

No, es completamente imposible predecir de antemano la cantidad exacta de pedidos que procesará cada estación.

Relación con Race Condition (Condición de Carrera):

Ambas estaciones de empaque (los dos procesos hijos) compiten de forma concurrente por extraer pedidos del mismo extremo de lectura (fd[0]) de la tubería compartida. Cuántos pedidos logre procesar cada una depende de factores dinámicos y no deterministas del sistema operativo, tales como:

- Las decisiones del planificador de procesos (CPU Scheduler).
- La asignación de ráfagas de tiempo de CPU (time slices / quantum).
- La latencia o tiempo de servicio que le toma a cada estación procesar su pedido actual antes de volver a solicitar el siguiente.
- Interrupciones del hardware y carga general del sistema.

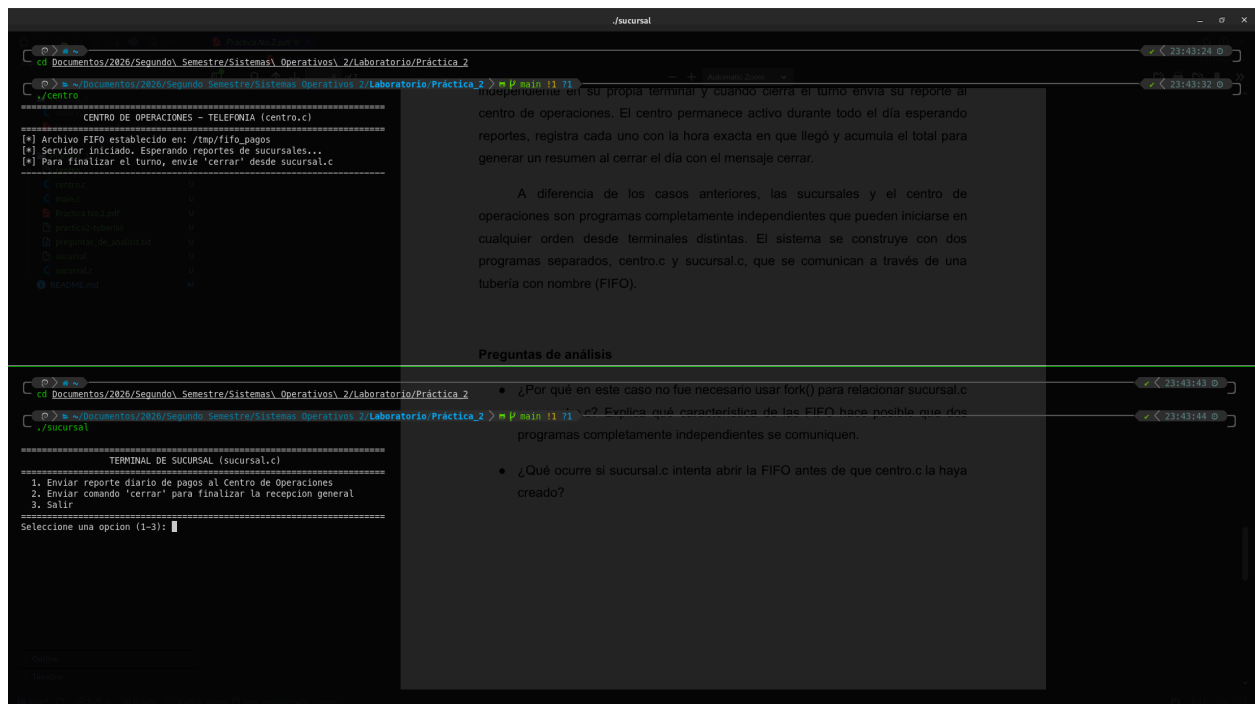
Esta dependencia del orden y sincronización temporal de eventos concurrentes es la definición formal de una Condición de Carrera (Race Condition). En cada ejecución del programa, la distribución de pedidos entre la Estación 1 y la Estación 2 variará de forma no determinista.

- **¿Qué pasaría si una de las estaciones no cerrará el extremo de escritura que heredó del coordinador?**

El programa sufriría un interbloqueo (Deadlock) permanente al final del turno. El sistema operativo gestiona una cuenta de referencias (reference count) para cada extremo de una tubería. La llamada al sistema read() solo devuelve 0 (EOF) cuando el búfer de la tubería está vacío Y TODOS los descriptores de escritura abiertos para esa tubería en TODO el sistema operativo han sido cerrados.

Al hacer fork(), ambos hijos heredan una copia exacta del descriptor de escritura (fd[1]) del padre. Si una de las estaciones olvida cerrar su copia:

- ### Problema 3 (Tuberías con FIFO)




```
./sucursal

[*] Archivo FIFO establecido en: /tmp/fifo_pagos
[*] Servidor iniciado. Esperando reportes de sucursales...
[*] Para finalizar el turno, envíe 'cerrar' desde sucursal.c

[2026-09-02 23:44:23] Reporte #01 recibido:
Sucursal: Quetzaltenango
Monto reportado: Q. 8500.50
Total acumulado: Q. 8500.50

[2026-09-02 23:44:39] Reporte #02 recibido:
Sucursal: Guatemala
Monto reportado: Q. 4000.00
Total acumulado: Q. 12500.50

[2026-09-02 23:44:47] >>> MENSAJE DE CIERRE RECIBIDO <<<

=====
RESUMEN CONSOLIDADO AL CIERRE DEL DIA
=====
Total de reportes recibidos: 2
Monto total de pagos del día: Q. 12500.50
Estado: FIFO cerrada y removida exitosamente.
=====

C:\Documents\2020\Segundo Semestre\Sistemas Operativos 2\Laboratorio\Práctica_2> P main 1 71
Preguntas de análisis

[*] Conectando con el Centro de Operaciones (/tmp/fifo_pagos)...
[*] Transmisión completada con éxito:
Contenido: "Guatemala|4000.00"
Bytes enviados: 17

=====
TERMINAL DE SUCURSAL (sucursal.c)
=====
1. Enviar reporte diario de pagos al Centro de Operaciones
2. Enviar comando 'cerrar' para finalizar la recepción general
3. Salir
=====
Seleccione una opción (1-3): 2

[*] Conectando con el Centro de Operaciones (/tmp/fifo_pagos)...
[*] Transmisión completada con éxito:
Contenido: "cerrar"
Bytes enviados: 6

=====
TERMINAL DE SUCURSAL (sucursal.c)
=====
1. Enviar reporte diario de pagos al Centro de Operaciones
2. Enviar comando 'cerrar' para finalizar la recepción general
3. Salir
=====
Seleccione una opción (1-3):
```

independiente en su propia terminal y cuando cierra el turno envía su reporte al centro de operaciones. El centro permanece activo durante todo el día esperando reportes, registra cada uno con la hora exacta en que llegó y acumula el total para generar un resumen al cerrar el día con el mensaje cerrar.

A diferencia de los casos anteriores, las sucursales y el centro de operaciones son programas completamente independientes que pueden iniciarse en cualquier orden desde terminales distintas. El sistema se construye con dos programas separados, centro.c y sucursal.c, que se comunican a través de una tubería con nombre (FIFO).

- ¿Por qué en este caso no fue necesario usar fork() para relacionar sucursal.c con centro.c? Explica qué característica de las FIFO hace posible que dos programas completamente independientes se comuniquen.
- ¿Qué ocurre si sucursal.c intenta abrir la FIFO antes de que centro.c la haya creado?

Preguntas de análisis

- **¿Por qué en este caso no fue necesario usar fork() para relacionar sucursal.c con centro.c? Explica qué característica de las FIFO hace posible que dos programas completamente independientes se comuniquen.**

No fue necesario usar fork() porque las FIFOs (también conocidas como Named Pipes o tuberías con nombre) poseen una presencia física y visible en el sistema de archivos a través de un nodo de archivo especial (i-nodo de tipo FIFO / p).

Diferencia fundamental con los pipes anónimos:

- Un pipe anónimo creado con pipe() carece de nombre o entrada en el sistema de archivos; reside únicamente en la memoria volátil del kernel y solo puede ser compartido mediante la tabla de descriptores de archivos heredada de un proceso ancestro a sus descendientes a través de fork().
- Una FIFO creada con mkfifo(pathname, mode) tiene una ruta de acceso en el sistema de archivos.

Cualquier proceso completamente ajeno, independiente y sin relación de parentesco genealógico puede acceder al canal de comunicación simplemente llamando a la función open() con dicha ruta, siempre que cuente con los permisos de lectura/escritura correspondientes en el sistema.

- **¿Qué ocurre si sucursal.c intenta abrir la FIFO antes de que centro.c la haya creado?**

Si el archivo especial FIFO aún no ha sido creado en el sistema de archivos mediante mkfifo():

- La llamada al sistema open(FIFO_PATH, O_WRONLY) en sucursal.c fallará inmediatamente.

- `open()` retornará `-1` y la variable global `errno` se establecerá con el código de error `ENOENT` (cuyo significado textual es "No such file or directory" / "No existe el fichero o el directorio").
- Si el programa no maneja adecuadamente este retorno de error, intentará escribir sobre un descriptor inválido (`-1`), provocando que `write()` falle con el error `EBADF` ("Bad file descriptor") o el cierre abrupto del programa.

Comportamiento si la FIFO YA FUE CREADA pero `centro.c` no la ha abierto para lectura:

Por el estándar POSIX, una llamada a `open(FIFO_PATH, O_WRONLY)` en modo bloqueante estándar se SUSPENDERÁ (se quedará en espera) hasta que otro proceso (como `centro.c`) abra el mismo archivo FIFO en modo lectura (`O_RDONLY` u `O_RDWR`), momento en el cual ambos procesos completan la apertura y pueden iniciar la transferencia de datos.

Link del repositorio

[Ingresar a la carpeta "Práctica_2"](#)